

Prepare data *for machine learning.*

Give every row a meaning.

Turn messy records into inputs you can trust.

🕒 18-22 min core reading · 20-30 min practice · 12 short sections

A model learns from the table you give it. Before choosing an algorithm, make sure each row means something, the inputs could exist at prediction time, and the evaluation data stays separate.

WHAT YOU WILL UNDERSTAND

The complete path: raw viewing records → checked rows → separate datasets → fitted preprocessing → model-ready numbers.

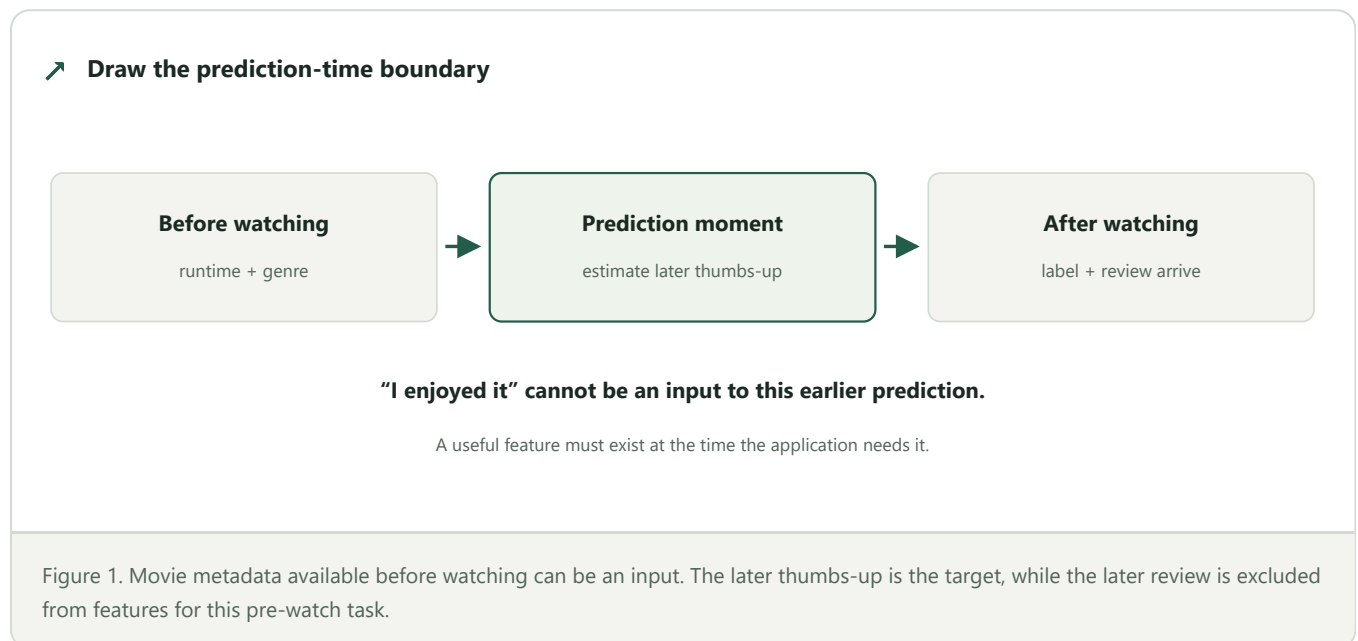
Our example: prepare data to predict whether a viewer will like a movie. This chapter prepares the inputs; Chapter 13 introduces supervised models.

Reading path: follow the core explanations, try the workbench, and run the Python workbook. The expandable notes add depth when you need it.

1. Decide what one row means

Chapter 11 selected movies using explicit rules. To learn from experience, we now need examples of previous viewing events. **One row represents one viewer watching one movie.** A movie watched by two viewers produces two different events.

Our question is: "Before this viewer watches, can we predict their later thumbs-up?" The **label**, or target, is the recorded outcome: `liked = 1` for a thumbs-up and `0` for a thumbs-down. The **features** are the inputs available when making that prediction.



Column	Meaning	Role in this workbook
<code>event_id</code> , <code>viewer_id</code> , <code>movie_id</code>	Which event, person, and film	Tracking and splitting; excluded from inputs
<code>runtime_minutes</code>	Movie length, sometimes missing	Numeric feature
<code>genre</code>	One primary genre	Categorical feature
<code>liked</code>	Feedback collected after watching	Label, kept separately
<code>review_after</code>	Text written after watching	Excluded: unavailable at prediction time

Using that later review to predict the earlier decision would give the model information it will not have when deployed. This is **target leakage**. A column can be accurate and still be an invalid input.

Our two features are deliberately simple. Without viewer preferences or history, identical movie inputs cannot give different predictions for different viewers. This is a data-preparation exercise, not a complete personalized recommender. All records are synthetic.

2. Inspect before changing anything

A **schema** describes the expected fields and types. A **data contract** also specifies acceptable values and meanings: runtime is in whole minutes, IDs follow a defined format, and the label is blank, `0`, or `1`. Keep an unchanged raw copy so corrections can be traced.

The workbook begins with 13 raw rows. Inspect types, missing counts, ranges, category spellings, unique event IDs, and label counts. For example, `" Adventure "` and `"ADVENTURE"` should become the same genre; `"105"` must become a number before arithmetic.

Raw value	Our explicit policy
<code>" Adventure "</code>	Trim whitespace; lowercase to <code>adventure</code>
<code>"105"</code> minutes	Parse to integer 105
Blank runtime	Preserve as missing, not zero
<code>"-5"</code> minutes	Quarantine the invalid record
Blank <code>liked</code>	Exclude from labeled examples; never assume dislike

Quarantine means set a record aside with its location and reason for investigation. The workbook accepts observed runtimes from 1 to 600 minutes as its declared scope. That limit is a chosen contract, not a universal definition of a movie.

▼ Go deeper: unusual values, units, and sampling bias

An **outlier** is unusual relative to other values; it is not automatically wrong. A 200-minute movie can be legitimate, while a negative runtime violates the meaning of duration. Investigate provenance and units before deleting values. A runtime of 2 might mean two minutes or a mistaken hours-to-minutes conversion; the number alone cannot establish which. Data-driven outlier thresholds must be learned on training data, not chosen from the entire dataset.

Sampling bias concerns who or what enters the data. Feedback from people who chose to watch a movie does not represent everybody who could have watched it. People who leave ratings may also differ from those who do not. Cleaning cannot fix this exposure and selection problem by itself. Keep the prediction population explicit, examine coverage, and record how labels were collected. Missing feedback is not a negative preference.

3. Resolve duplicates without deleting real events

A repeated movie ID is normal: different viewers can watch the same film. A repeated **event ID** needs investigation. In this dataset, the second copy of E03 has the same normalized values, so retaining one copy avoids counting the event twice.

If valid records share an event ID but disagree, our policy quarantines all of them. Choosing whichever row happens to appear first would silently invent a resolution. A second viewing with its own event ID remains a legitimate separate observation.

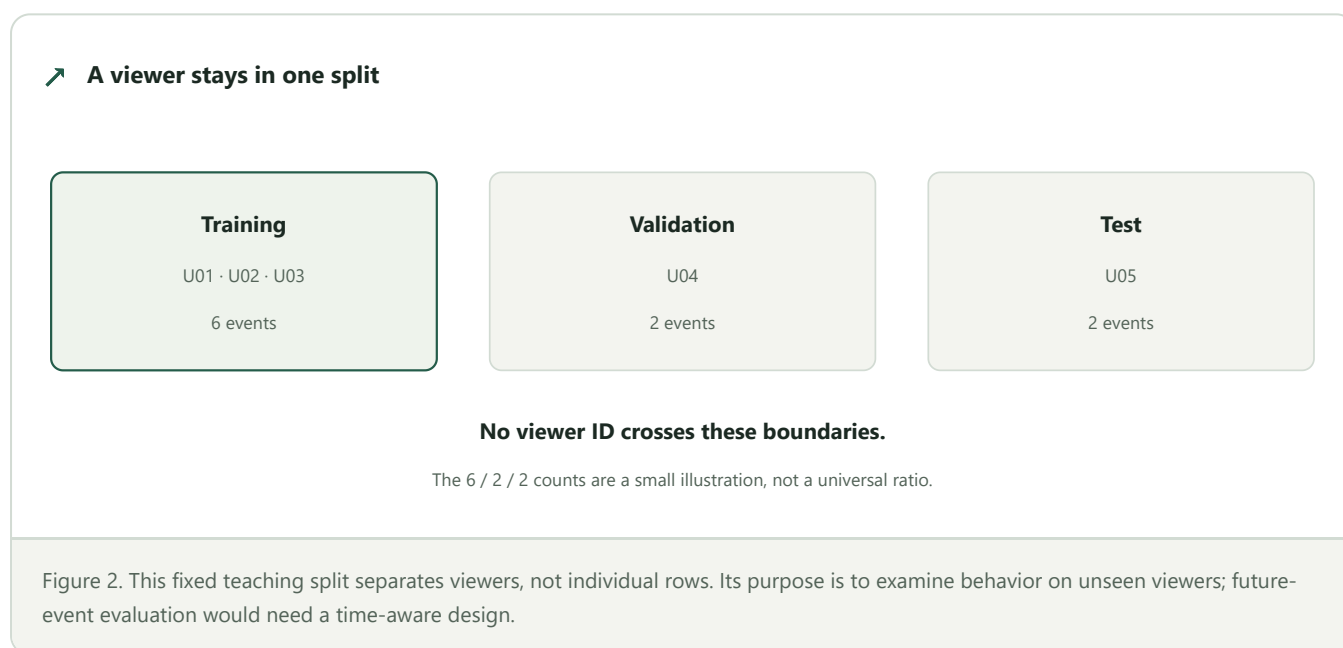
After fixed checks, the audit reads: **13 raw rows** → **10 labeled events**, with one duplicate copy removed, one negative-runtime row quarantined, and one unlabeled event set aside. The original missing runtimes remain missing at this stage.

These per-record contracts and identity checks use no estimated averages or fitted model. They can run before splitting. Decisions learned from the dataset-such as percentile cutoffs, feature selection, or a category vocabulary-belong inside the training process.

4. Split according to the question you will evaluate

The **training set** teaches the model and preprocessing. The **validation set** helps choose settings and compare approaches. The **test set** is reserved for evaluating the chosen approach. Repeatedly changing your system after seeing test results turns that set into tuning data.

Our evaluation question concerns **viewers absent from training**. All events from one viewer therefore stay in one split. Randomly shuffling individual rows could put the same viewer in both training and test, answering a different question.



This small plan puts U01-U03 in training, U04 in validation, and U05 in test. The 6/2/2 event counts are a teaching example, not a recommended ratio for every project. Two held-out events cannot support a reliable performance estimate.

▼ Go deeper: random, grouped, and time-based evaluation

Choose the split to match deployment. A random row split can suit independent observations. A grouped split keeps related observations-such as a viewer's events-together. To predict future behavior, split by time and construct historical features using only information available before each event. This workbook has no timestamps and does not demonstrate time-based evaluation. To evaluate unseen movies, consider grouping by movie; keeping both new viewers and new movies separate requires a more careful design.

Stratification tries to preserve class proportions; it does not by itself prevent group overlap or future information. In **cross-validation**, you rotate which training folds are held out, fitting preprocessing afresh inside each fold. Keep any final test set untouched. See scikit-learn's [cross-validation and splitting guide](#).

5. Fill missing values without inventing facts

Imputation supplies a replacement value when a chosen model needs numeric input. Our training rows contain four observed runtimes: 90, 105, 115, and 140 minutes. Their median is the midpoint of the two middle values:

$$\text{training median} = \frac{105+115}{2} = 110.$$

E04 and E06 have missing runtimes. We use 110 in their feature calculations and add a **missing indicator**:

`runtime_missing = 1` when the original value was absent, otherwise 0. The raw runtime stays missing.

This replacement makes a vector usable; it does not establish that the movie lasts 110 minutes. Chapter 11's hard time-limit filter still cannot certify a film whose actual runtime is unknown. Keep modeling values separate from verified catalog facts.

Mean imputation would use $(90 + 105 + 115 + 140) / 4 = 112.5$. The workbench lets you compare both methods. Median imputation is less affected by extreme observed values, but neither method recovers the missing truth.

▼ Go deeper: missingness is a modeling decision

Dropping every incomplete row can shrink the dataset and change who it represents. Filling a constant, mean, median, or a learned estimate makes different assumptions. A missing indicator preserves information about absence but does not automatically remove bias. More elaborate approaches can use other features to estimate missing values, and some estimators support missing values directly. See scikit-learn's [imputation guide](#).

If every training runtime is missing, there is no observed training median to calculate. The workbook stops and asks for an explicit alternative policy; it never borrows the test-set median. Missing labels are a separate problem: inventing a label creates invented supervision. Our pipeline sets those events aside rather than using feature imputation on the target.

6. Encode categories and scale numbers

Most basic estimators expect a numeric **feature matrix**, X , with one row per example and one column per feature. The corresponding labels form y . Keep their ordering aligned: row 4 of X must still match label 4 of y .

For genre, **one-hot encoding** creates separate indicator columns. An adventure event has 1 in `genre=adventure` and 0 in the other genre columns. Assigning arbitrary numbers such as comedy = 1 and drama = 2 would introduce an order or distance that the genre names do not imply.

Our training vocabulary has five genres. We reserve a sixth, `genre=__unknown__`, for unseen genres. Documentary appears only in held-out data, so it activates that bucket. The column order and count stay fixed.

For runtime, **standardization** subtracts the training mean and divides by the training standard deviation, both computed after filling missing training values:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

With median filling, the six training runtimes are 105, 140, 115, 110, 90, 110. Their mean is about 111.6667 and their population standard deviation is about 14.9071. E04's filled runtime becomes $(110 - 111.6667) / 14.9071 \approx -0.1118$. A negative standardized value means below the training mean, not a negative duration.

▼ Go deeper: which transformations help which models?

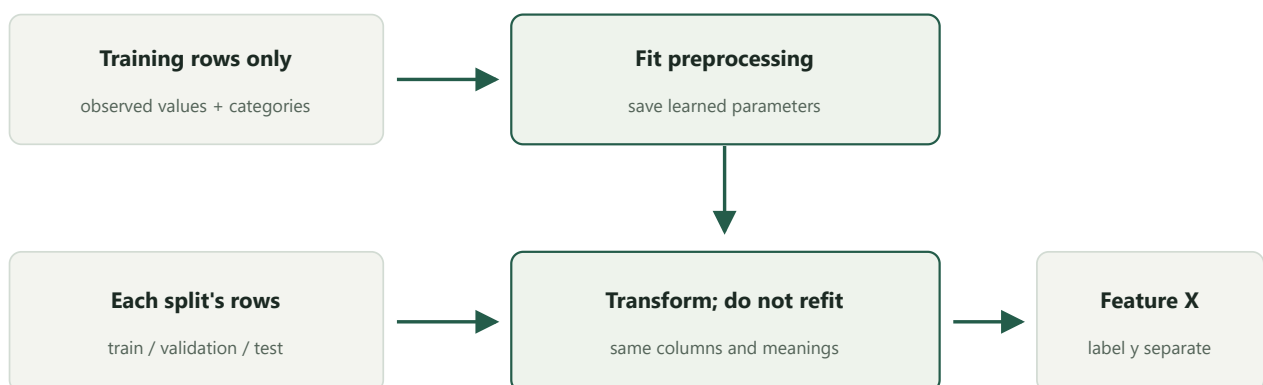
Scaling matters when an algorithm responds to feature magnitude, distance, or penalties. It is often useful for distance-based methods and regularized linear models; ordinary decision-tree splits generally do not require it. Standardization changes location and scale, not the distribution into a bell curve. This workbook divides squared deviations by the number of training rows, using population standard deviation; Chapter 7's sample estimate has a different purpose. A constant column uses a scale of 1 to avoid division by zero.

Min-max scaling uses a training range; robust scaling uses less outlier-sensitive statistics. Ordered categories can sometimes use an explicit ordinal mapping, while unordered categories need a suitable encoding. Our reserved unknown column is a deliberate convention, not the default behavior of every encoder. Fit and reuse the full transformation policy consistently. See scikit-learn's [preprocessing guide](#).

7. Separate fitting from transformation

Fit means learn preprocessing values from training rows: the replacement value, mean, standard deviation, and genre vocabulary. **Transform** means apply those already learned values to a row. A preprocessing step can be fitted even before a predictive model exists.

➤ Learn once from training. Reuse everywhere.



A new prediction uses the same saved transformation.

Figure 3. Training data determines fill values, scaling statistics, and the category vocabulary. Apply the frozen transformation to every split without fitting it again on held-out data.

Training, validation, test, and future inputs must use the same fitted transformation. Computing a separate test-set mean would both peek at held-out data and put that set into a different coordinate system.

Our pipeline order is **fill runtime** → **standardize runtime** → **add the missing flag and genre indicators**. The resulting training matrix has shape **6 × 8**: two runtime-related features and six genre columns. Validation and test each have shape **2 × 8**. IDs, labels, and later reviews stay outside X .

```
state = fit_preprocessor(splits["train"], "median")
train = transform_rows(splits["train"], state)
validation = transform_rows(splits["validation"], state)
test = transform_rows(splits["test"], state)
```

Fitting on all rows first is **preprocessing leakage**, even if the labels were never used. This separation is also the central advice in scikit-learn's [common pitfalls guide](#).

▼ Go deeper: [feature engineering and reusable pipelines](#)

Feature engineering constructs useful inputs, such as a missing flag, genre indicators, or a viewer's average rating of previously watched comedies. For a historical average, "previously" must mean before that example's prediction time; using later ratings leaks future information. Choosing which features to retain must also happen within training and validation, not by optimizing on final test results.

A pandas **DataFrame** is a labeled table useful for inspection and manipulation. Scikit-learn's **ColumnTransformer** can apply different transformations to numeric and categorical columns; a **Pipeline** connects preprocessing with an estimator. These tools help keep fitted steps together, including inside cross-validation. The standard-library workbook exposes the same fit/transform boundary directly so you can inspect every calculation without installing those packages.

8. Explore the preparation workbench

Start with training-only median filling and inspect E04. Its missing flag is 1, its sci-fi indicator is 1, and the target stays separate. Select E07 to see documentary enter the unknown bucket.

Now select **All splits · leakage demonstration**. The median becomes **127.5**, and a documentary column appears. Nothing about the six training records changed; held-out data changed the learned preparation. This shows the mechanism of leakage, not a measured increase in model accuracy.

Same rows. Different preparation.

13 raw records become 10 labeled events: one duplicate, one invalid runtime, and one missing label are kept out of the training table. No predictive model runs here.

Group split: U01-U03 supply 6 training events; U04 supplies 2 validation events; U05 supplies 2 test events.

Strategy	Fit scope	Fill	Scaling mean	Columns
median	Train only	110	111.6667	8
mean	Train only	112.5000	112.5000	8
median	All (leaky)	127.5000	132.5000	9
mean	All (leaky)	133.7500	133.7500	9

Default E04: missing runtime becomes 110, then $\text{runtime_z} \approx -0.1118$. The missing flag is 1, the sci-fi column is 1, and other genre columns are 0. E07's documentary genre uses the unknown bucket under the training-only vocabulary.

The prepared training matrix has 6 rows and 8 columns. Validation and test each have 2 rows with those same columns. The all-splits modes intentionally demonstrate leakage; no performance score is measured.

Switch between mean and median, then reset. The JSON download includes fitted values, feature names, aligned matrices and labels, the cleaning audit, and split membership. A leaky download is explicitly marked `leakage_demo: true`. Use it only to inspect the mistake.

9. Run the Python workbook and check the boundary

Extract the six files into one folder. Python 3.10 or newer is sufficient; no external packages or API key are needed. Run these commands from that folder:

```
py prepare_data.py
py prepare_data.py --strategy mean
py prepare_data.py --leaky-demo
py -m unittest -v test_prepare_data.py
```

Use your working `python` or `python3` command if `py` is unavailable. The program reads `raw_watch_events.csv` and `split_plan.json` beside the script and prints JSON. It does not train a model or change the input files. The browser uses JavaScript with the same synthetic records; automated checks compare its calculations with Python.

The default output has fill value 110, eight feature names, and an audit of the excluded records. Invalid rows are quarantined and reported. An invalid file header or an unusable split plan stops the run with exit code 2; successful preparation exits 0.

Check properties, not just whether the script runs: no viewer appears in two splits; all matrices use the same columns; every number is finite; each label stays aligned with its event; changing held-out values cannot change training-only parameters or training X .

▼ Go deeper: make preparation reproducible

Save the exact raw snapshot, its source and collection dates, the split plan, preprocessing code version, learned state, and ordered feature names. A random seed alone does not identify a dataset or guarantee reproducibility. This example uses an explicit viewer plan rather than a random split. Its output records `pipeline_version` and the plan, but you must retain the original files too; that version string is not a content hash.

When a fitted model is deployed, send new records through the matching saved preprocessing state. Recomputing a median from each incoming batch changes the meaning of its inputs. Monitor missingness, unknown categories, and other distribution changes; investigate before deciding to refit. This is one reason data preparation is part of the model system rather than a disposable notebook step.

10. Practice with three small experiments

Exercise 1: a number is not a fact

Calculate the training median from 90, 105, 115, and 140. E04 has no known runtime. After imputation, can Chapter 11's app promise that this film fits a 120-minute limit?

▼ Reveal the reasoning

The median is 110. E04 gets that placeholder for feature calculations and a missing flag of 1. The actual runtime remains unknown, so the hard time-limit filter still cannot establish eligibility. Imputation is not factual verification.

Exercise 2: make the held-out data obviously different

In a copy of the CSV, change E07's runtime from 180 to 500. Run normal preparation and then `py prepare_data.py --leaky-demo --strategy mean`. Which training inputs should stay unchanged?

▼ Reveal the reasoning

With training-only preparation, every fitted value and every training feature vector stays unchanged. E07's own transformed runtime changes, as it should. In the leaky mean run, the pooled fill value and scaling statistics change, affecting training vectors too. The test suite checks this independence directly. Restore the CSV before the next exercise.

Exercise 3: move a review across the boundary

Change the text of E04's `review_after`, keeping all other values the same. Should its feature vector change? Then consider what happens if E04 alone moves into test while E03 from the same viewer remains in training.

▼ Reveal the reasoning

The review edit changes no feature: later review text is excluded from X. Splitting E03 and E04 apart would violate this chapter's unseen-viewer evaluation because both belong to U02. The workbook assigns whole viewers, so moving U02 in the plan moves both events together. A different evaluation question may need a different plan, not an accidental exception.

11. Check your understanding

01 You predict a thumbs-up before watching. Which column is an invalid feature?

- A. Runtime known before watching.
- B. The review written after watching.
- C. The movie genre known before watching.

Answer: B. The later review is unavailable at prediction time and may reveal the outcome. Using it would leak future information.

02 Where should you learn the median used to fill missing training runtimes?

- A. From the observed training runtimes only.
- B. From all rows so the estimate is more accurate.
- C. From whichever split gives the best test score.

Answer: A. Held-out values must not affect fitted preprocessing. Apply the training median unchanged to validation, test, and later inputs.

03 A new genre was absent from the training vocabulary. What does our pipeline do?

- A. Refit the vocabulary on the test set.
- B. Give it the same code as adventure.
- C. Activate the explicit unknown-genre column.

Answer: C. A reserved unknown bucket keeps the feature layout fixed. It does not imply that the model has learned how to handle this new genre well.

04 Why do all events from a viewer stay together in this split?

- A. Every ML task requires a viewer-group split.
- B. This evaluation question concerns unseen viewers.
- C. A fixed random seed guarantees independence.

Answer: B. The split should match the generalization question. Random event splits can put the same viewer on both sides; forecasting future events would require an appropriate time-based design.

05 The model input uses 110 for an unknown runtime. What can the movie app conclude?

- A. The movie is definitely 110 minutes long.
- B. The movie definitely satisfies a 120-minute limit.
- C. 110 is a modeling placeholder; the actual runtime remains unknown.

Answer: C. Imputation creates usable model input, not a verified catalog fact. Keep the original missing value and its indicator.

12. Keep the preparation story straight

You should now be able to explain this chain: **define the event and prediction time** → **inspect and check records** → **choose a deployment-relevant split** → **fit preprocessing on training only** → **reuse it everywhere** → **verify the boundaries**.

Before fitting a model, ask whether the label means what you intend, the inputs would exist at prediction time, and evaluation examples represent the question you care about. A clean-looking table is necessary work; it is not proof of representative data or a useful model.

▼ Quick-reference glossary

Term	Meaning here
Observation	One viewer-movie event
Feature / label	Available input / outcome to predict
Schema / contract	Expected structure / structure plus value rules and meaning
Quarantine	Set a problematic record aside with a reason
Group split	Keep all records of a chosen entity in one split
Imputation	Replace a missing value for a modeling calculation
Missing indicator	A feature recording that the original value was absent
One-hot encoding	Separate 0/1 columns for categories
Standardization	Subtract a fitted mean and divide by a fitted standard deviation
Fit / transform	Learn preparation values / apply the learned values
Leakage	Information enters learning that should be unavailable
Feature engineering	Construct useful inputs from available information

Continue with the primary references: scikit-learn's leakage guidance, missing-value methods, encoding and scaling, and evaluation splits. Our data, policies, diagrams, and numerical walkthrough are authored for this handbook.

Next: Chapter 13 - Supervised learning: regression and classification. Use prepared inputs to learn a numerical prediction or a class, then connect predictions to losses and decision boundaries.